

Aula modelo

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

2 de junho de 2026



1 SQLAlchemy

2 SQLAlchemy com FastAPI



1 SQLAlchemy

2 SQLAlchemy com FastAPI



ORM (Object Relational Mapping) é uma técnica para mapear tabelas do banco de dados para classes Python.

Sem ORM

- Escrever SQL manualmente
- Converter resultados para objetos

Com ORM

- Classes representam tabelas
- Objetos representam registros
- Métodos substituem comandos SQL simples

Benefícios

- Menos SQL repetitivo
- Código mais organizado
- Compatibilidade entre bancos

GeoPackage (SQLite)

```
from sqlalchemy import create_engine

engine = create_engine(
    "sqlite:///dados.gpkg"
)
```

PostGIS (PostgreSQL)

```
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg://"  
    "postgres:postgres"  
    "@localhost/geodb"
)
```

O restante do código ORM é praticamente o mesmo para ambos.

Maapeando uma Tabela para uma Classe



```
from sqlalchemy.orm import (
    DeclarativeBase,
    Mapped,
    mapped_column
)

class Base(DeclarativeBase):
    pass

class Cidade(Base):

    __tablename__ = "cidades"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    nome: Mapped[str]

    populacao: Mapped[int]
```

Correspondência

```
Base.metadata.create_all(  
    engine  
)
```

SQL equivalente:

```
CREATE TABLE cidades (  
    id INTEGER PRIMARY KEY,  
    nome TEXT,  
    populacao INTEGER  
)
```

O ORM gera a estrutura automaticamente.

```
from sqlalchemy.orm import Session

with Session(engine) as session:

    cidade = Cidade(
        nome="Curitiba",
        populacao=1774000
    )

    session.add(cidade)

    session.commit()
```

Equivalente SQL:

```
INSERT INTO cidades
(nome, populacao)
VALUES
('Curitiba', 1774000);
```

```
from sqlalchemy import select

with Session(engine) as session:

    stmt = select(Cidade)

    resultado = session.scalars(
        stmt
    )

    for cidade in resultado:
        print(cidade.nome)
```

Equivalente SQL:

```
SELECT *
FROM cidades;
```

```
from sqlalchemy import select

stmt = (
    select(Cidade)
    .where(
        Cidade.populacao > 1000000
    )
    .order_by(
        Cidade.nome
    )
)
```

Equivalente SQL:

```
SELECT *
FROM cidades
WHERE populacao > 1000000
ORDER BY nome;
```

GeoAlchemy2

```
from geoalchemy2 import Geometry

class Cidade(Base):

    __tablename__ = "cidades"

    id = mapped_column(
        primary_key=True
    )

    nome = mapped_column()

    geom = mapped_column(
        Geometry(
            "POINT",
            srid=4326
        )
    )
```

A coluna geom passa a ser tratada como uma geometria espacial pelo

GeoPackage

- Aplicações desktop
- Dados locais
- Arquivo único

PostGIS

- Múltiplos usuários
- Grandes volumes de dados
- Serviços web e APIs

SQLAlchemy + GeoAlchemy2

- Abstração do banco
- Código reutilizável
- Integração com FastAPI, Flask e GeoPandas



1 SQLAlchemy

2 SQLAlchemy com FastAPI

FastAPI é um framework moderno para construção de APIs REST em Python.

Integração típica

Cliente → FastAPI → SQLAlchemy → PostgreSQL/PostGIS

Vantagens

- APIs REST rapidamente
- Integração nativa com JSON
- Documentação automática (Swagger)
- Suporte a PostgreSQL e PostGIS

```
from sqlalchemy import create_engine

DATABASE_URL = (
    "postgresql+psycopg://"
    "postgres:postgres"
    "@localhost/geodb"
)

engine = create_engine(
    DATABASE_URL
)
```

Sessões do banco

```
from sqlalchemy.orm import Session

with Session(engine) as session:
    ...
```

```
from sqlalchemy.orm import (
    DeclarativeBase,
    mapped_column
)

class Base(DeclarativeBase):
    pass

class Cidade(Base):

    __tablename__ = "cidades"

    id = mapped_column(
        primary_key=True
    )

    nome = mapped_column()

    populacao = mapped_column()
```

A classe Cidade representa a tabela do banco.



```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def raiz():

    return {
        "mensagem":
            "API funcionando"
    }
```

Executar:

```
uvicorn app:app --reload
```

```
from sqlalchemy import select
from sqlalchemy.orm import Session

@app.get("/cidades")

def listar_cidades():

    with Session(engine) as session:

        stmt = select(Cidade)

        cidades = session.scalars(
            stmt
        ).all()

        return cidades
```

Endpoint:

/cidades



```
@app.post("/cidades")  
  
def criar_cidade():  
  
    with Session(engine) as session:  
  
        cidade = Cidade(  
            nome="Campinas",  
            populacao=1214000  
        )  
  
        session.add(cidade)  
  
        session.commit()  
  
    return {"status": "ok"}
```



```
from pydantic import BaseModel

class CidadeDTO(BaseModel):

    nome: str
    populacao: int

@app.post("/cidades")

def criar(cidade: CidadeDTO):

    print(cidade.nome)

    return {"status": "ok"}
```

Exemplo JSON:

```
{
  "nome": "Curitiba",
  "populacao": 1774000
}
```



```
@app.get("/cidades/{id}")

def buscar(id: int):

    with Session(engine) as session:

        cidade = session.get(
            Cidade,
            id
        )

    return cidade
```

Exemplos:

```
/cidades/1
/cidades/2
```

- Documentação gerada automaticamente
- Testes de API diretamente no navegador
- Visualização dos modelos Pydantic

Após iniciar a aplicação:

```
http://localhost:8000/docs
```

Permite executar:

- GET
- POST
- PUT
- DELETE

Sem escrever código cliente.

Cliente (Browser, Python, QGIS)



FastAPI



SQLAlchemy ORM



PostgreSQL / PostGIS

Benefícios

- Código organizado
- Menos SQL manual
- APIs REST modernas
- Integração com sistemas GIS